Proceedings of the 4th International Conference on Automation, Computing and Renewable Systems (ICACRS-2025)
IEEE Xplore Part number: CFP25CB5-ART; ISBN: 979-8-3315-4886-5

# A Comparative Analysis on the Differences Between Client-Side Databases and Server-Side Databases

Shayantan Bose
*Computer Science and Engineering*
SRM Institute of Science and Technology
*Hyderabad*, India
shayantan.bose.uni@gmail.com

Sathvik S Rao
*Computer Science and Engineering*
SRM Institute of Science and Technoogy
Bengaluru, India
sathvik2416@gmail.com

Ansh Ezhava
*Computer Science and Engineering*
SRM Institute of Science and Technology
Ahmedabad, India
anshdpsg@gmai

Dr. J. Shajeena
*Assistant Professor Department of Computer Science and Engineering*
SRM Institute of Science and Technology, Tiruchirapalli campus, India
shajeenaelmo@gmail.com
ORCID 0009-0008-8332-2588

2025 4th International Conference on Automation, Computing and Renewable Systems (ICACRS) | 979-8-3315-4886-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICACRS67045.2025.11324165

***Abstract*—Databases play an important role in creating any contemporary software or web-based application. Historically databases are of various kinds, such as relational, non-relational, and cloud databases. With the HTML5 standardization, local or session-based storage databases have become more popular that are directly included in web browsers. While client-side storage has its advantages like offline usage and server computation offloading, it also has its own drawbacks, such as the need for local space dependency. Historic server-side databases still enjoy scalability advantages.**

**Within this review paper, we examine the performance of client-side local storage databases (IndexedDB and LocalStorage) in Google Chrome 134. We also compare it with server-side databases, such as PostgreSQL, MySQL, and the non-relational database MongoDB. Our test cases consist of the most prevalent create, read, update, and delete (CRUD) operations. Recognizing these differences in performance is important, since optimizing where data is located will yield a better-performing application with seamless user interaction. This comparison hopes to assist developers in choosing the correct database, since the selection ultimately lies with the size and needs of the application.**

***Keywords—HTML5, Server-side Database, Client-side Database, LocalStorage, IndexedDB, MongoDB, PostgreSQL, MySQL, Chromium-based Application, Pgbench.***

## 1. INTRODUCTION

With the rise of complex web applications, developers face a critical architectural decision: whether to store data on the client-side or the server-side. While both approaches have known benefits , there is a lack of recent, direct performance benchmarks comparing modern client-side databases (like IndexedDB) against common server-side counterparts (like PostgreSQL, MySQL, and MongoDB) for basic CRUD operations. This gap makes it difficult to make informed choices to optimize application speed and user experience.

Until the past decade a lot of Database Management System's architecture required for the database to be stored in a mainframe as it usually took a lot of resources for processing. The quest for offloading these processing power begin from then. Now things have changed, meaning there's better Database Management System architectures and other innovations have also taken place. There are larger storage devices which are more reliable and the modern consumer end workstations are capable enough to store up to a certain amount of data.

It's noticed that there are avid number of web applications being developed rather than native applications. Due to the nature of web application being cross platform, a disadvantage arises that being its reliability on local storage data which push developers to develop more native applications with dedication databases stored in servers. While developing web applications, local storage is important as they can store and create user's web browsing activities. For example, a social media site will generate and store data about user preference and settings, cached user profile data, etc.

With the introduction of HTML5, web browsers were required to run client side applications the same way a desktop application runs, so it can avoid the ineffectiveness and network connectivity issues that comes with server side databases. HTML5 introduced IndexedDB as a part of web technologies. IndexedDB consists of multiple databases for the same domain while storing that data in LevelDB format. For browsers which are based on Chromium, data is stored in key-value pair. For the browsers that support incognito mode (private mode), the data is stored in the primary memory while in normal browsing mode the data is stored in the secondary memory and then brought to primary memory when needed.[7]

This paper aims to compare between various types of server databases to client side databases. Client side database include IndexedDB and LocalStorage. Server side database include MongoDB, MySQL and PostgreSQL. The benchmarking has gathered numerous data from various test sets, on which the most common create, read, update and delete (CRUD) operations were performed. The client-side databases were tested on only one browser that is Google Chrome 134.

PostgreSQL was tested with its in-built benchmarking tool, pgbench. MySQL and MongoDB were tested through a Node.js environment. [1]

### 1. *Motivation and Research Questions*

With recent trend in web-based applications, the importance of deciding to choose between which data will be stored in client-side database and server-side database is crucial. Many of these applications deals with user data and application data to run them, optimizing where the data is stored in will result in a faster running application with smooth user interaction. There has been a lot of development in the recent years regarding client-side databases and server-side databases, this review paper aims to look at the answers to few of these questions.[1] [6]

RQ1. In what aspects does the client-based database outperform the server-side database, vice versa?

RQ2. When should each of the database be selected to be used?

## B. *Contribution*

In summary, this review paper tries to achieve two major contributions as follows.

- Relevancy and Use cases of the databases with respect to the type of data being stored.
- Discussion of hybrid systems that combines client-server database for optimal performance.
- Validating our data through tests and reaching a final conclusion.

The remainder of this paper is structured as follows. Section 2 gives a background on the various types of DBMS of client-side databases and server-side databases. Section 3 describes the test suite environment and what tests were conducted to analyze the performance. Section 4 draws the results and analysis derived from the tests conducted. Section 5 discusses the future work and motivation. Lastly, Section 6 gives the conclusion and summarizes our contributions.[1] [2]

# II. Background and Related Work

## A. *Background*

### 1. *Client-Side Databases.*

A client side database system is a database that is stored locally in the client's machine. Client side databases are used in order to make efficient systems. Processing and fetching data from the server requires a lot of processing power and good network connectivity, hence a lot of applications store user data that do not need to be globally declared in the client's system itself to avoid these issues. In the past few decades because of the developments in larger and reliable storage options and powerful workstations, a lot of the data is stored locally. Client-side databases do have their own advantages and disadvantages. Few of its advantages are as follows. [4]

1) It doesn't require any network. Unlike server-side databases or even cloud-based databases, client-side database requires no consistent network connection to fetch data or update data. Therefore, there is no need to factor in the user's internet speed, connectivity issues or general reliability. There is also no need to factor in the costs of uploading data or modifying data. [10]

2) Have full physical control over all the data. Since the data is not stored in a different location, the users have full access over all their data that is physically stored in their device itself. There is decreased risk of intrusion and tampering of the user's data due to it being stored locally on their device. Users can also provide permissions on what to and who can view the data that's stored locally.

Few of its disadvantages are as follows.

1) Dependency on client's resources. The performance of a client based database will largely depend upon the resources that the client is providing and also the browser's capabilities. Certain tasks may be slowed down purely because of the older or less powerful devices. [10]

2) Data Synchronization Challenges. If the application is built on a hybrid system where it's using both client-side database and server-side database, maintaining consistent data across all fronts is a challenge, especially in scenarios where frequent updates or offline usage takes place. Hence conflict resolution and synchronizations techniques should properly be implemented. [10]

### b. *IndexedDB*

IndexedDB is a client side storage solution directly integrated into web browsers which was introduced by W3C as a standard when HTML5 was released particularly valuable for modern offline-first application. It's an object oriented non-relational databases which is schemeless and it stores data in LevelDB format in most browsers. In IndexedDB data is stored in key-value pairs but keys can be referred as properties of objects that are stored in value. LevelDB stores its data with two key extensions which are Idb and log. The rest of the files aid Idb and log to keep a version and exchange information. The .log file keeps a track of the data changed such as data insertion, modification, and deletion. Since IndexedDB doesn't use SQL as the query language, retrieval process of data is faster than those database which operate on SQL. Currently, the latest version of IndexedDB provided by W3C is IndexedDB API version 3.0 that was released in 3.0 [15]. One of the main notable characteristics of IndexedDB is it's automatic database creation i.e. if a database doesn't exist an attempt is made to automatically create the database. Even though a lot of the client's data can be stored in IndexedDB, browsers still limit the amount of storage IndexedDB uses to limit the consumption of the resources. Below is Table 1. which shows the different storage limit that each browser has set up for storing data in IndexedDB. [15]

TABLE 1. Browser-Specific IndexedDB Limits.

| Browser | Approximate Limit |
|---|---|
| Firefox | ~2 GB (desktop) or ~ 5 MB initial (mobile) |
| Chrome / Chromium | Up to ~80% of free disk, per origin cap |
| Safari (iOS) | ~1 GB per origin (variable) |
| Edge | Similar to Chrome's 80% of free space |
| iOS Safari | Typically 1 GB, can be less on older iOS |
| Android Chrome | Similar to desktop Chrome |

Table 1. Table that shows the browser-specific storage limits for IndexedDB

Fig 1. Shows the code of how a single record is fetched by key in IndexedDB in JavaScript.

**1) How to fetch data?**

```
const transaction = db.transaction(['yourObjectName'], 'readonly')
const objectStore = transaction.objectStore('yourObjectName')
const getRequest = objectStore.get(yourKeyValue)
getRequest.onsuccess = (event) => {
  const data = event.target.result
  if (data) {
    console.log('Fetched data:', data)
  } else {
    console.log('No data found with key:', yourKeyValue)
  }
}

getRequest.onerror = (event) => {
  console.error('Error fetching data:', event.target.error)
}
```

Fig 1. Code of how a single record is fetched by key.

Fig 2. Is the code that shows how to delete a single record by key in IndexedDB in JavaScript.

**2) How to delete a single record by key?**

```
const transaction = db.transaction(['yourObjectName'], 'readwrite');
const objectStore = transaction.objectStore('yourObjectName');
const deleteRequest = objectStore.delete(yourKeyValue);
deleteRequest.onsuccess = (event) => {
  console.log('Record with key', yourKeyValue, 'deleted successfully.');
};

deleteRequest.onerror = (event) => {
  console.error('Error deleting record:', event.target.error);
};

transaction.oncomplete = (event) => {
  console.log('Transaction completed (delete).');
};

transaction.onerror = (event) => {
  console.error('Transaction error (delete):', event.target.error);
};
```

Fig 2. Code of how to delete a single record by key.

Fig 3. Is the code that shows how to update a single record via a known key in IndexedDB in JavaScript.

**3) How to update record with known key?**

```
const transaction = db.transaction(['yourObjectName'], 'readwrite');
const objectStore = transaction.objectStore('yourObjectName');

const updatedData = {
  id: yourKeyValue,
  propertyName1: newValue1,
  propertyName2: newValue2,
};

const putRequest = objectStore.put(updatedData);

putRequest.onsuccess = (event) => {
  console.log('Record with key', yourKeyValue, 'updated successfully.');
  console.log('New key (if auto-generated):', event.target.result);
};

putRequest.onerror = (event) => {
  console.error('Error updating record:', event.target.error);
};

transaction.oncomplete = (event) => {
  console.log('Transaction completed (update).');
};

transaction.onerror = (event) => {
  console.error('Transaction error (update):', event.target.error);
};
```

Fig 3. Code of how a single record is update with known key.

*c. LocalStorage.*

Local Storage just like IndexedDB is a client-side storage solution that is directly integrated within multiple web-browsers. It's used to store user data that are used frequently so it can offload any performance overhead that the server side databases may suffer from if we fetch this type of data every time when we want to use it. Since LocalStorage is a part of WebAPI it can use some of its property like window interface that allows web applications to retain information across browsing section. The primary advantage of LocalStorage is its ability to maintain application state and user setting without constantly relying on server-side data retrieval. It provides various benefits like the ability to remember client-specific settings such as user themes, language options or other data like JWT, etc. that is cached frequently. Unlike IndexedDB the data stored via LocalStorage is in string format, which is extremely inefficient compared to the format that IndexedDB uses which is a key-value pair in LevelDB format. Also unlike IndexedDB, LocalStorage blocks of the main thread because its synchronous and can slow down the browser. Fig. 4, Fig. 5, and Fig. 6, are code snippets that show how to fetch, delete and update data in LocalStorage. [5] [11]

Fig 4. This code snippet demonstrates how to fetch data from LocalStorage in JavaScript

**1) How to fetch data?**

```
const key = 'yourKey';
const storedValue = localStorage.getItem(key);
```

Fig 4. Code that shows how to fetch data in LocalStorage.

Fig 5. The code snippet attached is to show how to delete data from LocalStorage in JavaScript.

**2) How to delete data?**

```
const keyToDelete = "yourKeyToDelete";
localStorage.removeItem(keyToDelete);
```

Fig 5. Code that shows how to delete data in LocalStorage.

Fig 6. Is the code snippet that shows how to update data from LocalStorage in JavaScript.

**3) How to update data?**

```
const keyToUpdate = 'yourKeyToUpdate';
const newValue = 'yourNewValue';
localStorage.setItem(keyToUpdate, newValue);
```

Fig 6. Code that shows how to update data in LocalStorage.

*B. Server Side Databases.*

Server-side databases form the essential core of most modern applications, providing persistent and a centralized storage for critical data. These systems, usually residing in dedicated or shared servers, facilitate reliable and concurrent access to data for numerous users and client applications, concurrently, a feature that is largely contrasting to the client-side based database or local storage. Unlike client-side storage, server-side databases provide sophisticated features and implementations, such as robust transaction management, high data integrity, scalable architecture, and much more features crucial for complex application needs. The three widely adopted server-side databases are **MySQL**, **PostgreSQL**, **MongoDB**. We will be doing a comparative analysis in-be-

tween the given databases aiming to provide clear understanding the strengths and weaknesses of each one and to guide informed database selection. [3] [6]

*a. MongoDB.*

MongoDB is a NoSQL database, meaning it is not based on the traditional database model and rather follows a document oriented model. The data is stored in the form of flexible, JSON-like documents, making it easily adaptable to evolving, new data structures. The database is adapted and designed to provide high performance and easy scalability, for applications with large amount of unstructured or semi-structured data. - Data is stored in documents, which are similar to JSON objects, allowing nested data structures for flexible schemas. MongoDB doesn't enforce a rigid schema, allowing flexibilities which are beneficial for applications with rapidly changing data requirements. MongoDB can distribute data across multiple servers enabling it to handle massive datasets and high traffic loads. It supports replication, creating multiple copies of data across different servers, ensuring data redundancy and high availability. [8] [9]Some of its use cases are as follows

1) Web Applications. Flexibility and Scalability make it suitable for web applications.

2) Real Time Analytics. MongoDB can handle large volumes of real-time data for analytics purposes.

3) Content Management Systems. The flexible schema is well-suited for managing diverse content types.

4) Big Data Application. Its ability to handle large volumes of unstructured data is very useful for big data analytics.

MongoDB has its own strengths and weakness. Few of its strengths are as follows.

**1) Strengths**

1.1) Flexibility and Agility.

1.2) Scalability and high performance for certain applications.

1.3) Ease of development.

**2) Weakness**

2.1) It faces consistency challenges.

2.2) Less mature transaction support compared to relational database.

2.3) Can consume a lot of memory.

We'll also look at few of the different data manipulation techniques we can use. These are as follows.

**1) How to fetch data?**

db.collection.find({ userId: 123 }).toArray();. This command fetches all the documents that has the userId as 123.

db.collection.findOne({ userId: 123 });. This fetches a single document, which comes first when there are multiple for the given same parameter.

**2) How to update data?**

db.collection.updateOne({ userId: 123 }, { $set: { status: "active" } });.
Updates the status field of the user with userId 123 in the document.

db.collection.updateMany({ status: "pending" }, { $set: { status: "active" } });.

Updates all the with status as "pending" to "active for all documents following the condition.

**3) How to delete data.**

db.collection.deleteOne({ userId: 123 });. Deletes a single document, with the userId as 123.

db.collection.deleteMany({ status: "inactive" });. Deletes multiple documents with the status as "inactive".

*b. PostgreSQL.*

PostgreSQL is a powerful, open-source relational database management system. It follows the SQL standard and offers advanced features for data integrity and complex queries. Some of its key features are that it ensures data integrity through ACID properties and provides robust transaction management for complex operations. It supports a wide range of data types, including JSON, arrays, and geometric data. Allows the creation of custom functions, operators, and data types through extensions. Provides powerful SQL features for complex data retrieval and analysis. PostgreSQL has a lot of use cases, some of them are as follows. Complex Transactional Applications. Suitable for applications that require strong data consistency and reliability. Data Warehousing. Its advance queries make it suitable for data analysis and reporting. Geographic Information Systems Supports spatial datatypes for GIS applications. Web applications needing strong data integrity.[12] Just like other types of databases PostgreSQL also has its strengths and weaknesses. Few of them as follows.

**1) Strengths.**

1.1) Data integrity and reliability.

1.2) Extensibility and powerful features.

1.3) Strong community support.

**2) Weakness.**

2.1) Can be resource intensive.

2.2) Steeper learning curve than MySQL.

Here's few data manipulation techniques that's used in PostgreSQL.

**1) How to fetch data?**

SELECT * FROM users WHERE user_id = 123;.
Fetches all the rows with the matching user_id.

SELECT name, email FROM users
WHERE status = 'active';.
Fetches specific columns.

**2) How to delete data?**

DELETE FROM WHERE category = 'outdated';.
Delete one or more rows based on condition.

**3) How to update data?**

UPDATE products SET price = price * 1.1
WHERE category = 'electronics';.
Update multiple rows based on condition.

*c. MySQL.*

MySQL is the most popular, open-source RBDMS known for its ease of use and performance. Widely used in web applications and content management systems. Few of its key features are as follows. MySQL is relatively easy to install and configure. Optimized for read-heavy applications, making it suitable for websites with high user traffic. Supports replication of data and clustering for high availability. Active supportive community and ample resources for support. Various different storage engines, each one with its unique characteristics. Few of its use cases are as follows. [8]

1) Web Application. A popular choice for web applications due to its performance and ease of use.

2) Content Management System. Widely used for platforms such as WordPress that manage stored content.

3) E-Commerce. Used for online stores, especially for read heavy platforms.

Like other databases that are reviewed in this paper MySQL also has its own strengths and weaknesses. They are as follows:-

**1) Strengths.**

1.1) Speed and performant.

1.2) Ease of Use.

1.3) Widely used.

**2) Weaknesses.**

2.1) Less feature rich than PostgreSQL

2.2) Some limitations in scalability due to complex architectures.

Here's few data manipulation techniques that's used in PostgreSQL.

**1) How to fetch data?**

```
SELECT * FROM users WHERE user_id = 123;.
```
Fetches all the columns for matching rows.

```
SELECT name, email FROM users
WHERE status = 'active';.
```
Fetches specific columns for matching rows.

**2) How to delete data?**

```
DELETE FROM products
WHERE category = 'outdated';.
```
Delete one or many rows based on condition.

**3) How to update data?**

```
UPDATE products SET price = price * 1.1
WHERE  category = 'electronics';.
```
Update multiple rows based on condition.

*C. Differences between Client Side Database and Server Side Database.*

TABLE 2. COMPARISON BETWEEN CLIENT-SIDE AND SERVER-SIDE DATABASES

| Feature | Client-Side Database | Server-Side Database |
|---|---|---|
| Location & Execution | Embedded within the user's web browser or mobile application. Data is stored and managed locally. | The data is sent and stored in a remote device that is accessible over the network. |
| Primary Purpose | It helps in enhancing the user experience through offline functionality, caching frequently accessed data, storing user preferences and lightweight data persistence. | Manage large volumes of data, of various users, in structured or unstructured format, and being accessible from anywhere. |
| Typical Use Cases | Offline desktop, mobile, and web applications, browser extensions, storing user settings, caching content for faster loading, etc. | Web applications, content management platform, e - commerce websites, business intelligence, etc. |
| Performance and Characteristics | Limited by client's device's hardware, including but not limiting to CPU, GPU, RAM, storage, etc. Faster for minimal operations and latency is negligible for actions carried out locally. | Dependent on the hardware where the server is hosted upon, network bandwidth, and database optimization. |
| Security Considerations | Less secure as compared to server-side database, as data is stored in the client's device, that can pose threat due to possibility of unauthorized access. | Offer robust security features such as authentication, authorization, encryption and access control through roles and permissions. Server administrators are responsible for setting up and maintaining security measures. |
| Scalability Limitations | Limited by the storage and processing power of the client's device hardware. Not designed for handling massive datasets. | Highly scalable, vertically & horizontally. Can handle large datasets and high traffic volumes of users or requests. |
| Data consistency & Integrity | Data cannot be made consistent across devices as the client side databases are limited to a particular browsers or the whole device at most. | Implement ACID properties to ensure data consistency and integrity across devices. Offer robust mechanisms for data validation and co |

Table 2. A table that compares between client-side database and server-side database.

*D. RELATED WORK*

This section of the paper discusses the existing studies and research of the comparision between the performances of client-side databases versus server-side databases, and the analysis of their different storage mechanisms that they have integrated.

*a. Client-side Databases versus Server-side Databases.*

1. Kimak and Ellman (2013) performed a comparision between client-side databases mainly IndexedDB versus server-side database MySQL. Their work highlights the importance of client-side databases with the introduction of HTML5. While their work analysed the difference in performance of client-side databases in two browsers, Chrome (v.22) and FireFox (v.15), we have tested the client-side databases on one browser only that is Chrome 134. They have also tested WebSQL which we have not, as its no longer widely used as much. In addition to client-side databases, we have also include three server-

side databases (MongoDB, PostgreSQL and MySQL) in our performance analysis.[1]

2. In the past, several works have examined IndexedDB and client-side databases. Paligu's (2022) work investigated IndexedDB from a digital forensics perspective, highlightings its importance in storing data in web-based applications and the challenges it presents forensic investigators.[4]
3. Arefipour and Mozahhebi (2013) compared IndexedDB, by focusing on developer's concern regarding security and performances [5].

## III. Test Suite Environment

Client-side databases that were tested are, IndexedDB and LocalStorage and were tested with JavaScript objects. Both the database support Firefox and Google Chrome but they were tested on Google Chrome 134. The server-side databases that were tested were PostgreSQL, MongoDB and MySQL. The server-side databases were tested locally with a sample dataset of half a million that was randomly generated. PostgreSQL was test with the in-built benchmarking tool pgbench and the other two server-side databases were tested through a Node.js environment.[1] [2]
The hardware specifications of the device that is used for testing is:
•CPU – Intel(R) Core(TM) i5-10300H (8) at 4.50 GHz.
GPU - NVIDIA GeForce GTX 1650 Mobile / Max Q [Discrete] / Intel UHD Graphics at 1.05 GHz [Integrated]

- RAM – 15.45 GiB
- Storage – 463.74 GiB
- OS – Arch Linux

The software specifications of the test-suite that is used for testing is:

- Node.js Version – 22.20.0
- Database –
  - pg version – 17.6
  - MySQL version – 8.4
  - MongoDB version – 8.0

### A. *Benchmark for Client-Side Databases.*

The benchmark consisted of a bunch of queries performed on a dataset of size 5000 records/index. A random JavaScript object was generated and then inserted. The dataset used for both were pragmatically generated with varying data for each insertion. The structure of the object consisted of an id, value (content) and timestamp. The value property had a predictable pattern, which provided variability without being statistically random. The benchmark evaluates the time need to insert the generated given dataset, retrieve a record with key, retrieve a record with index (not possible for LocalStorage as it stores data in simple key-value structure and no internal data structure for indexing), update the value of items for the first 10% of the keys and delete a subset of items starting from the middle of the keys (roughly the last 50%). Below is table 3 and fig 7 that shows the difference in performance between LocalStorage and IndexedDB. [5]

TABLE 3. Performance comparison of IndexedDB and LocalStorage.

| Feature | LocalStorage | IndexedDB |
|---|---|---|
| Data Size | 5,000 items | 5,000 records |
| Operation | Time (ms) | Time (ms) |
| Insertion | 40.449 | 237.589 |
| Retrieval (Key) | 6.599 | 1.401 |
| Retrieval (Index) | N/A (No Indexing) | 0.136 |
| Update | 5.334 | 109.235 |
| Deletion | 16.887 | 54.345 |

Table 3. Performance comparision table between the client-side databases.

As shown in Table 3, LocalStorage outperformed IndexedDB in insertion, update, and deletion operations.

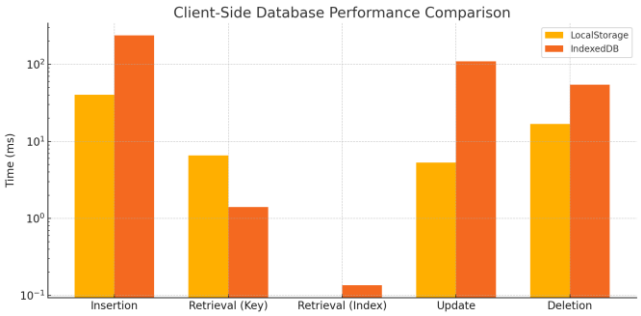


Fig 7. Figure of a graph that shows performance comparision between the client-side databases.

As shown in Fig 7, LocalStorage outperformed IndexedDB in insertion, update, and deletion operations but it lacks an operation to retreive data via an index.

### B *Benchmark for Server-Side Databases.*

Server-side databases that were tested are PostgreSQL, MongoDB and MySQL. PostgreSQL's performance was tested with its in-built benchmarking tool pgbench. In pgbench the main factor that determines the dataset size is scale factor. The dataset generated is a synthetic dataset generated by pgbench with a scaling factor of 5 i.e., 500,000 rows. The benchmark consisted of the time to insert records, retrieval by key, retrieval by index, update and deletion (which were performed on a subset of the dataset instead of the whole dataset). Table 4 provides the overall context for our analysis. For example, it immediately illustrates that MySQL and IndexedDB had exceptionally slow insertion times, while offered the fastest insertions. We will now break down these findings, looking first at the client-side databases.[8]

TABLE 4. Performance comparison of PostgreSQL, MySQL and IndexedDB.

| Feature | PostgreSQL | MySQL | MongoDB |
|---|---|---|---|
| Data Size | ~500,000 rows | ~500,000 rows | ~500,000 documents |
| Operation | Time (ms) | Time (ms) | Time (ms) |
| Insertion | 2353.325 | 60752 | 1358 |
| Retrieval (Key) | 0.745 | 3 | 1 |

| | | | |
|---|---|---|---|
| Retrieval (Index) | 3.868 | 6 | 7 |
| Update | 28.514 | 1 | 3 |
| Deletion | 25.374 | 1 | 1401 |

Table 4. Performance comparison table between the server-side databases.

Based on the server-side database comparison in Table 4 (which compares PostgreSQL, MySQL, and MongoDB with ~500,000 entries ), a clear performance trade-off is evident. MongoDB's document-oriented model led to the fastest insertion time (1358 ms) , making it suitable for applications with large, unstructured data. In sharp contrast, MySQL was exceptionally fast at update and deletion operations , recording a time of just 1 ms for both. However, MySQL was the slowest at insertion, and MongoDB was the slowest at deletion. PostgreSQL emerged as the strongest performer in data retrieval, posting the fastest times for both key-based (0.745 ms) and index-based (3.868 ms) lookups. These results, gathered on a personal device, indicate that the optimal database choice depends heavily on the application's primary function—whether it's writing new data, modifying existing data, or reading data quickly (Fig.8).

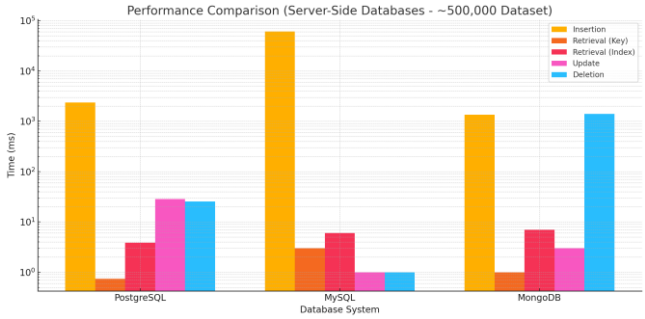


Fig 8. Figure of a graph that shows performance comparison between the server-side databases.

## IV. Results and Analysis

This section presents the findings from the benchmark tests conducted on both client-side and server-side databases. As shown in Table 4 and Table 3, the performance results vary dramatically across the different systems and operations. The use of a logarithmic scale in the graph highlights these significant disparities, particularly in insertion and deletion times.

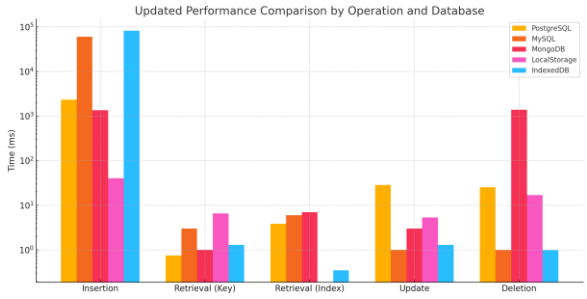


Fig 9. Figure of a graph that shows performance comparison between the databases that were tested.

When discussing Figure 9, it is critical to remember that the datasets were different. The client-side databases were tested with 5,000 items , while the server-side databases were tested with 500,000 rows. Therefore, this graph does not compare their raw speed directly but rather shows the performance characteristics of each system under its own test load. Furthermore, all tests were run on a single personal device, so results may vary based on hardware and server configuration.

### *A Client-Side Databases*

The benchmark results comparing the client-side databases, namely, LocalStorage and IndexedDB, for a dataset size of 5,000 items are as follows:

1. **Insertion:** LocalStorage demonstrated significantly faster insertion times compared to IndexedDB.
2. **Retrieval (Key):** IndexedDB outperformed LocalStorage on retrieving data using a provided key.
3. **Retrieval (Index):** Only IndexedDB has the feature of retrieval by index, which has demonstrated better performance than retrieval by key. LocalStorage doesn't have this capability.
4. **Update:** LocalStorage was faster in updating items as compared to IndexedDB.
5. **Deletion:** LocalStorage had faster deletion times compared to IndexedDB.

### *B. Server-Side Databases*

The benchmark results for three main common server-side databases, MySQL, PostgreSQL and MongoDB with a dataset of approximately 500,000 rows, in case of MySQL and PostgreSQL and 500,000 documents in case of MongoDB are as follows:

1. **Insertion:** MongoDB showed the fastest insertion time, followed by PostgreSQL, with MySQL being the slowest of them all.
2. **Retrieval (Key):** PostgreSQL was the fastest at retrieving documents using given key, with MongoDB being the second, and MySQL being the slowest.
3. **Retrieval (Index):** PostgreSQL was the fastest at retrieval by index, followed by MongoDB and MySQL, being not that far apart from each other.
4. **Update:** MySQL demonstrated the fastest update time, followed by MongoDB and PostgreSQL.
5. **Deletion:** MySQL showed the fastest deletion time, followed by PostgreSQL and after a huge gap, MongoDB.[9]

## V. Discussion of Analysis

This section will interpret and provide insights to the performance and characteristics of each type of database [13]. It is critical to note that these results are based on a single test machine and one web browser, Google Chrome 134. As noted in our future work, performance can vary significantly based on hardware configurations, operating systems, and other browsers (like Firefox or Safari). Therefore, these results should be seen as a specific benchmark rather than a universal rule [14].

**Client-Side Databases:**

- LocalStorage's synchronous nature and simple key-value storage structure makes it viable and faster for basic operations such as insert, delete, update and deletion on smaller datasets.
- IndexedDB's asynchronous operations and structured data storage with indexing capabilities make it

more efficient for retrieval operations on larger stored datasets, and complex queries.

**Server-Side Databases:**

- MongoDB's document-oriented model allows for faster insertion and retrieval of data, making it suitable for large complex applications with unstructured or semi-structured data.
- MySQL is optimized for update and deletion operations, having incredibly fast times for both operations.
- PostgreSQL, while being robust and feature rich, showed slower performance in some operations compared to MongoDB and MySQL, but advanced features ensuring data integrity makes up for the performance degradations.
- It is important to note that the testing for all these databases was done on a personal device, thus the results can vary based on the hardware and the environment the test is being run on.[1] [2]

## VI. Motivation

- Test across more web browsers beyond Google Chrome 134.
- Evaluate performance on different operating systems or hardware configurations, also including a wider range of databases.
- Conducting a more detailed investigation into hybrid systems that combine client and server-side databases.
- Analyze the impact of database version updates or browser updates on performance.
- Highlight the crucial importance of selecting appropriate client-side vs server-side databases for modern web applications to optimize performance.

## VII. Conclusion and Future Work

On the client-side, where IndexedDB excels at retrieval actions, using indexes, client-side local storage provides speed for basic actions such as inserting, updating and deleting smaller data sets. On the server-side, MongoDB excels in insertions, giving exceptionally less times, while MySQL provides better performance on update and delete actions, and PostgreSQL offers effective retrieval times based on key or indexes. Ultimately, the choice of platform, client or server, and the choice of the type of storage totally depends on the scale and requirement of the application, and focusing on the type of operation to be carried out.

## References

[1] *S. Kimak and J. Ellman, "Performance Testing and Comparison of Client Side Databases Versus Server Side," in 2013 PGNet, 2013, pp. 1-7.*

[2] Przychodzień, Łukasz, Dominika Radwan, and Grzegorz Kozieł. "Comparative analysis of selected databases on the example of a proprietary web application." *Journal of Computer Sciences Institute* 28 (2023): 190-196.

[3] A. Delis and N. Roussopoulos, "Performance and Scalability of Client-Server Database Architectures," in *Proceedings of the 18th VLDB Conference*, Vancouver, British Columbia, Canada, 1992.

[4] F. Paligu, "Investigation of IndexedDB persistent storage for digital forensics," Doctor of Philosophy (Digital and Cyber Forensic Science), Sam Houston State University, Huntsville, Texas, 2022.

[5] S. Arefipour and M. Mozahhebi, "Comparison of IndexedDB and SQLite Based on Developers' Concerns," Bachelor of Science Thesis in Software Engineering and Management, University of Gothenburg, 2013.

[6] B. Marina, I. Memon, and R. Naz, "Comparative analysis of Cloud database, remote database, and traditional database," *International Journal of Computer Science and Information Security (IJCSIS)*, vol. 14, no. 10, October 2016.

[7] B. Jeong, S. Lee, and J. Park, "MIC: Memory analysis of IndexedDB data on Chromium-based applications," *Forensic Science International: Digital Investigation*, vol. 50, 2024, p. 301809.

[8] D. Damodaran B, S. Salim, and S. M. Vargese, "PERFORMANCE EVALUATION OF MYSQL AND MONGODB DATABASES," *International Journal on Cybernetics & Informatics (IJCI)*, vol. 5, no. 2, April 2016.

[9] A. Makris, K. Tserpes, G. Spiliopoulos, D. Zissis, and D. Anagnostopoulos, "MongoDB Vs PostgreSQL: A comparative study on performance aspects," *Geoinformatica*, vol. 25, no. 2, pp. 243-268, 2021.

[10] Cloud Storage vs. Local Storage: 19 Pros & Cons," Carbide Secure. [Online]. Available: https://carbidesecure.com/resources/cloud-storage-vs-local-storage-19-pros-and-cons/. [*Accessed: 31 Mar. 2025*].

[11] Camden, Raymond. *Client-side data storage: keeping it local*. " O'Reilly Media, Inc.", 2015.

[12] "PostgreSQL Documentation," PostgreSQL. [Online]. Available: https://www.postgresql.org/docs/. [*Accessed: 1 Apr. 2025*].

[13] "IndexedDB API 3.0," World Wide Web Consortium (W3C). [Online]. Available: https://w3c.github.io/IndexedDB/. [*Accessed: 2 Apr. 2025*].

[14] "IndexedDB Max Storage Limit," RxDB. [Online]. Available: https://rxdb.info/articles/indexeddb-max-storage-limit.html. [*Accessed: 1 Apr. 2025*].

[15] Paligu, Furkan, and Cihan Varol. "Browser forensic investigations of whatsapp web utilizing indexeddb persistent storage." *Future Internet* 12, no. 11 (2020): 184.